

LibraryCard — Friday Questions

1. **Why must the card number get its value in the member initialization list?**

`number` is declared as `const int`, and a `const` data member cannot be assigned after the object exists — every assignment to it in the constructor body is a compile error. The member-initializer list is the one place where the member is *born with* its value instead of being default-constructed and then overwritten, so `number(nextNumber++)` must appear there. A side effect is that the compiler, not my memory, is what guarantees the number never changes afterwards.

2. **Why does `printReport(const LibraryCard& card)` force you to mark some member functions `const`?**

The `const` on the reference promise the function will not modify the card it receives, so inside `printReport` the parameter is treated as a `const` object. C++ only lets a `const` object call *const member functions*, because only those promise (and the compiler enforces) that they do not change the object's state. If `getHolder()` or `print()` were not marked `const`, the calls `card.getHolder()` and `card.print()` would fail to compile, since those functions could in principle write to the card. So marking every read-only member function `const` is not optional decoration — it is what makes the card usable through a read-only reference at all.

3. **At the end of `main`, `lobby[1]`, `lobby[0]` and `ana` all close. In what order, and why?**

The order is `lobby[1]` (card 5006) first, then `lobby[0]` (card 5005), and `ana` (card 5001) last of those three. Automatic (local) objects are destroyed in *reverse order of construction* when their block ends. `ana` was constructed first of all the locals, so it is destroyed last; the array `lobby` was constructed last, so it is destroyed first, and its elements go in reverse index order (`lobby[1]` before `lobby[0]`). Between them, `x` (5003) and `ben` (5002) close in the same reverse pattern, which is exactly why the output ends with 5006, 5005, 5003, 5002, 5001.

4. **How can `LibraryCard::activeCards()` be called when no card exists?**

Because `activeCards()` is a `static` member function: it belongs to the class itself rather than to any one card object, so it does not need an instance to be called — `LibraryCard::activeCards()` invokes it directly on the class. What it reads is also static: the `count` data member lives once for the whole program, not inside each card, so it exists even at the start of `main` when zero cards have been created. Constructors increment `count` and the destructor decrements it, so the static function always reports the current number of live cards.

5. **Reflection.**

This exercise tied the whole week's lecture material together in one small class, and building it forced me to understand each feature as a tool rather than syntax. The biggest “aha” was `const`: I initially thought of it as labeling data that never changes, but the `printReport`

question showed me `const` is really a promise about member functions — a contract between the class and its callers. Delegating constructors and the private static helpers were the other win; seeing all three construction paths funnel through one body made the “write each rule once” rule feel natural instead of dogmatic. The deleted copy operations and the destructor output also made object lifetime concrete for the first time — I could *see* the reverse destruction order in the program’s output instead of just being told it exists.

Discussing the homework with the other members of my home group helped too: walking through why `temp`’s destructor fires early, and comparing our different answers for the must-not-compile checks, surfaced details I had glossed over (for example, why the string constructor has to be `explicit` rather than just non-copyable). My one point of feedback for the week is that the assignment’s habit of stating behaviors without naming the C++ feature was frustrating while I was stuck but probably the most valuable part afterwards — being forced to pick the right mechanism myself is what made the features stick. I would not change that, but a short checklist of “which week-6 feature could produce this behavior” after the due date might help students verify they found the intended solution.